Containers for HPC: Pre-built Scientific Software with E4S

Apptainer · Slurm · MPI · E4S 26.06 on Midway3

Research Computing Center, University of Chicago

John Crerar Library, Kathleen A. Zar Room · Tuesday, October 27, 2026 · 3–5 PM

Today: three parts, all hands-on



0:10

Part 1

Containers on shared HPC systems

Why Apptainer, not Docker; the rootless model



0:30

Part 2

Pull, inspect, run

E4S images, what is inside, Slurm batch jobs



1:10

Part 3a

Swap MPI at launch

e4s-cl: container binary, host MPI, full speed



1:35

Part 3b

Add software to an image

e4s-alc: YAML → definition file → new image

Out of scope: building images from source — see RCC's general container workshops.

Before we start: a compute node and the labs

```
$ ssh <cnetid>@midway3.rcc.uchicago.edu
$ source /project/rcc/workshops/e4s/env.sh
$ sinteractive -c 4 -t 2:00:00
# ... now on a compute node: set up again
$ source /project/rcc/workshops/e4s/env.sh
$ cp -r $E4S_WORKSHOP/labs $E4S_SCRATCH/e4s-labs
$ cd $E4S_SCRATCH/e4s-labs
$ less LAB_GUIDE.md
```

- **env.sh** loads apptainer/1.4.1, puts e4s-cl and e4s-alc on your PATH, sets $E4S_IMAGE
- **Image cache → scratch:** a single E4S image would fill your 30 GB home quota
- **Run containers on compute nodes,** not login nodes

HANDS-ON **LAB_GUIDE.md · Part 0 · 5 minutes**

Why pre-built scientific software?

"Your software environment should not be your first research project."

Dozens

of interdependent libraries behind one scientific application

× compiler × MPI × GPU

every combination has to be built and tested again

Days–weeks

to configure a complex HPC software stack correctly



E4S answers with pre-built, co-tested images.
Pull one file, and LAMMPS, GROMACS, PETSc, hypre, Trilinos and 600+ other packages are ready, built consistently against one compiler and MPI.

Message adapted from S. Shende, "E4S" tutorial, MVAPICH Users Group (MUG'26), Aug 2026.

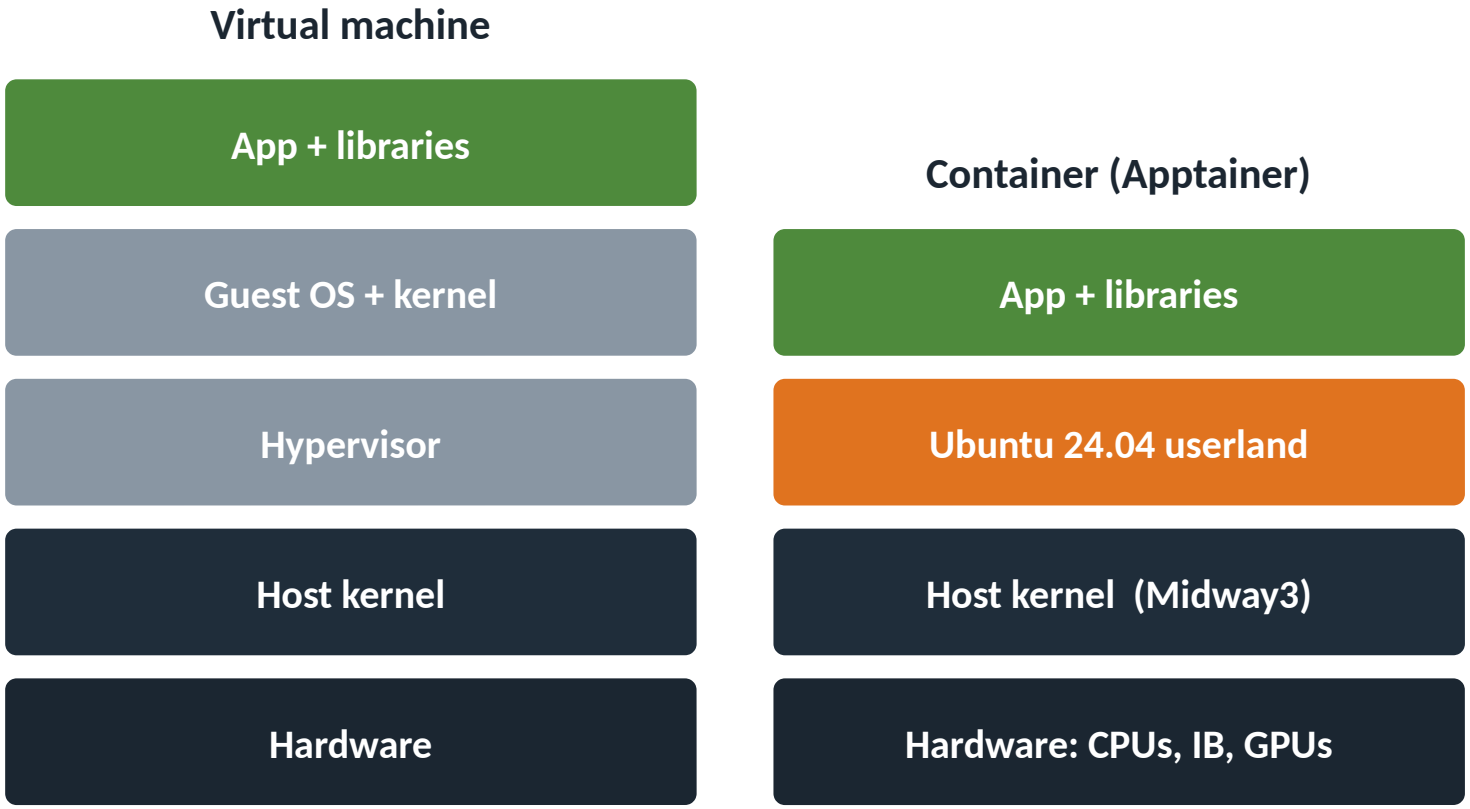
PART 1 · 20 MIN

Containers on shared HPC systems



What a container is, why HPC centers use Apptainer instead of Docker, and what “rootless” means in practice.

A container is a different userland on the same kernel



- **Inside:** files — OS libraries, compilers, MPI, applications
- **Outside:** kernel, CPUs, memory, network, GPUs, file systems
- **No hypervisor,** so no performance tax: processes run directly on the hardware
- **Portable,** with a caveat: the kernel and drivers come from the host

After S. Shende, "What are containers / Hypervisors and Containers", MUG'24.

Why Apptainer, not Docker, on a shared cluster

	Docker	Apptainer
Who runs it	A root-owned daemon starts your container	**You do; it is just your process**
Privileges	docker group ≈ root on the node	**Same UID inside and out; no escalation**
Image format	Layers under /var/lib/docker	**One read-only .sif file on GPFS**
Scheduler	Outside Slurm's view	**srun / sbatch launch it like any program**
MPI and GPUs	Network namespace gets in the way	**Host network and devices; --nv / --rocm**

Docker images still matter: apptainer pull docker://… converts them, so everything on Docker Hub (including E4S) is usable.

The rootless execution model



Same user

id -un inside = outside. Files you create are yours; Slurm accounting just works.



Read-only image

The .sif is mounted read-only. --writable-tmpfs adds a throwaway layer in RAM.



Your files are mounted

$HOME, /tmp and the current directory appear automatically; add more with -B.



Environment flows in

Host variables pass through (needed by MPI); --cleanenv starts from a minimal set.



Root only in builds

--fakeroot maps you to root inside a build namespace, so apt-get works; the host is untouched.

Apptainer in six commands

```
$ apptainer pull e4s.sif docker://ecpe4s/e4s-cpu:26.06
$ apptainer exec e4s.sif <command>
$ apptainer shell e4s.sif
$ apptainer run e4s.sif
$ apptainer inspect e4s.sif
$ apptainer build --fakeroot new.sif recipe.def
```

- **pull:** Docker/OCI image → single SIF file
- **exec:** run one command inside
- **shell:** interactive shell inside
- **run:** the image's default action (runscript)
- **inspect:** labels, runscript, definition
- **build:** new image from a recipe (Part 3b)

```
# useful flags
-B /host:/ctr      bind a directory      --cleanenv       don't pass host env
--writable-tmpfs   scratch layer in RAM   --nv / --rocm   NVIDIA / AMD GPUs
```

Hands-on 1: poke at the E4S image

```
$ apptainer exec $E4S_IMAGE grep PRETTY_NAME /etc/os-release
PRETTY_NAME="Ubuntu 24.04.4 LTS"      # host: CentOS Linux 8
$ id -un; apptainer exec $E4S_IMAGE id -un
<cnetid> / <cnetid>                    # same user, same UID
$ apptainer exec $E4S_IMAGE touch /opt/hello
touch: cannot touch '/opt/hello': Read-only file system
$ apptainer exec --writable-tmpfs $E4S_IMAGE touch /opt/hello
$ apptainer exec $E4S_IMAGE touch ~/hi && ls ~/hi
$ apptainer exec --cleanenv $E4S_IMAGE env | wc -l
28                                     # instead of ~245
$ apptainer inspect --runscript $E4S_IMAGE
```

- A different Linux, the same kernel and hardware
- No root: you are you, inside and out
- The image never changes; your home does
- --cleanenv: tidy, but breaks MPI

HANDS-ON **LAB_GUIDE.md · Part 1 · 10 minutes**

PART 2 · 35 MIN

E4S images: pull, inspect, run under Slurm



Get an image, find out exactly what is inside it, and run an application as a batch job.

E4S: the Extreme-scale Scientific Software Stack

140+

HPC and AI products in the E4S catalog

26.06

current release, June 22, 2026

2 / yr

co-tested releases, frozen Spack environments

- **A curated Spack distribution:** math libraries, data & I/O, performance tools, programming models, AI frameworks, and applications
- **Delivered as binaries:** Docker/Apptainer images and a Spack build cache (cache.e4s.io)
- **26.06:** Spack 1.1.1 · CUDA 13.2 · ROCm 7.2.3 · oneAPI 2026.0 · Ubuntu 24.04 and Rocky 9 · x86_64, aarch64, ppc64le
- **Community-maintained,** led by Sameer Shende (U. Oregon); part of the High Performance Software Foundation

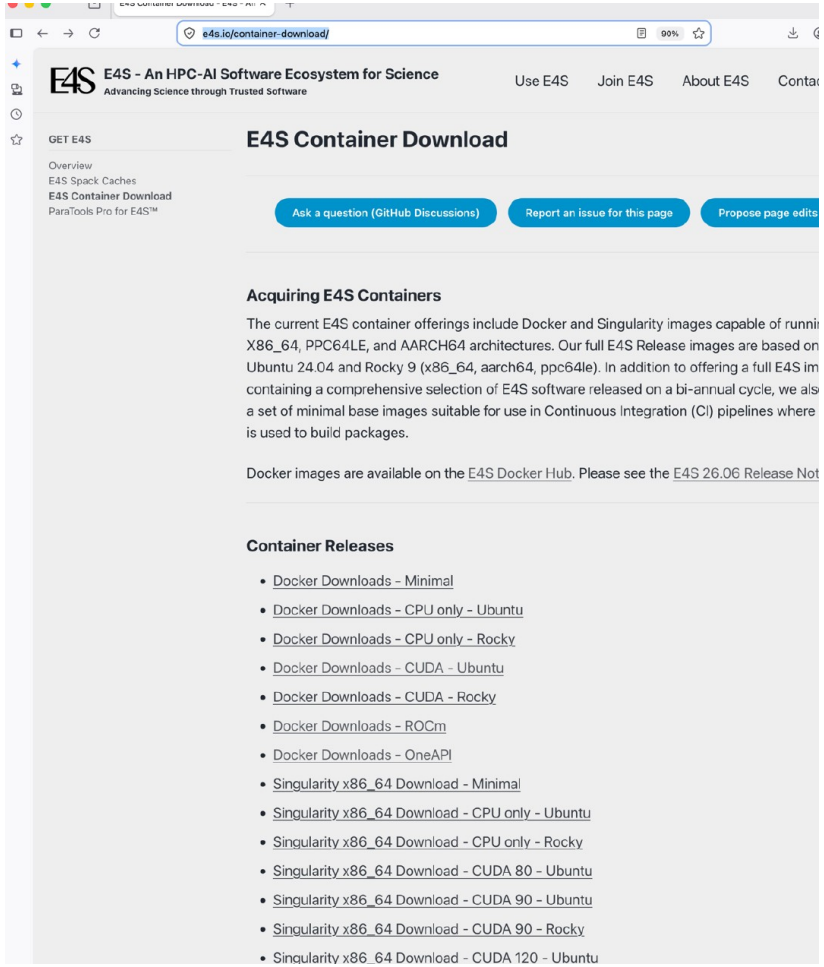
Sources: e4s.io product catalog; S. Shende, "E4S 26.06 release" slides, June 2026.

Which E4S image? Full, base, or cache-backed

Image	What is inside	Size	Today
Full (cpu, cuda, rocm, oneapi)	The whole stack: 600+ Spack packages, apps, tools	22–34 GB	**e4s-cpu 26.06, pre-pulled**
Base (e4s-base-*)	MPICH, CMake and a GPU SDK, for building your own	5–8 GB	—
Cache-backed (e4s-spack-cpu)	Spack + MPICH; install anything else from the build cache	1.5 GB	**You pull it**
Minimal (ecpe4s/ubuntu24.04)	Ubuntu + Spack, a starting point for e4s-alc	1.2 GB	**Part 3b base**

- **Where:** Docker Hub (docker://ecpe4s/…) or ready-made SIFs at e4s.io/container-download
- **Match your hardware:** cuda80 = A100, cuda90 = H100; Midway3 caslake nodes are CPU-only, hence e4s-cpu

Getting the images




```
# Docker Hub -> SIF (2-4 min for 1.5 GB)
$ apptainer pull e4s-spack-cpu_26.06.sif \
      docker://ecpe4s/e4s-spack-cpu:26.06

# Ready-made SIF from the E4S project
$ curl -LO https://oaciss.nic.uoregon.edu/\
      e4s/images/26.06/e4s-cpu-x86_64-26.06.sif
```

- **pull =** download layers into $APPTAINER_CACHEDIR, then squash into one SIF (CPU-heavy: use a compute node)
- **Docker Hub limit:** 100 anonymous pulls / hour / IP — a room of 30 is fine, a script in a loop is not

Screenshot: e4s.io/container-download, from S. Shende, "E4S 26.06 release" slides.

Inside e4s-cpu 26.06: 127 explicit, 685 total packages

Math libraries

PETSc · hypre · Trilinos · SUNDIALS · SuperLU-DIST · MFEM · SLEPc · STRUMPACK · Ginkgo · libCEED

Applications

LAMMPS · GROMACS · OpenFOAM · Quantum ESPRESSO · CP2K · WarpX · WRF · Nek5000 · Xyce · Laghos

Data & I/O

HDF5 · ADIOS2 · NetCDF · PnetCDF · Conduit · Ascent · zfp · SZ3 · Darshan

Programming models

MPICH · Open MPI · Kokkos · RAJA · Umpire · Legion · HPX · UPC++ · Chapel

Performance tools

TAU · HPCToolkit · PAPI · Caliper · Dyninst

E4S tools

e4s-cl 1.0.9 · e4s-alc 1.0.3 · Spack 1.1.1

How to find out what is inside

```
$ apptainer inspect $E4S_IMAGE
$ apptainer shell --writable-tmpfs $E4S_IMAGE
Apptainer> source /spack/share/spack/setup-env.sh
Apptainer> spack find -x           # 127 packages
Apptainer> spack find -v petsc@3.25.1
petsc@3.25.1 ~cuda+double+fortran~hypre+mpi
             ~superlu-dist ...
Apptainer> spack find -d hypre     # dependencies
Apptainer> spack location -i lammps
```



Spack wants a lock file.
The image is read-only, so add --writable-tmpfs or spack find fails.

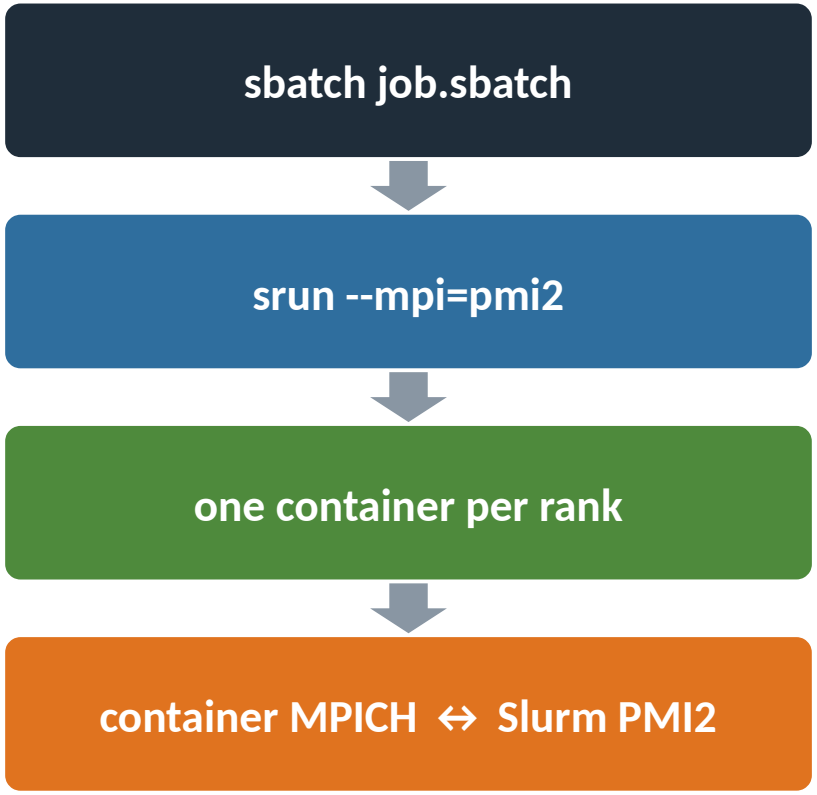


Read the variants.
+ is on, ~ is off. This PETSc has no SuperLU_DIST and no hypre.



Prefer shell/exec over run.
run sources the image's bashrc, which writes into $HOME.

Running containers under Slurm




```
#!/bin/bash
#SBATCH --nodes=1 --ntasks=8
#SBATCH --time=00:10:00
source $E4S_WORKSHOP/env.sh

LMP=$(apptainer exec --writable-tmpfs \
  $E4S_IMAGE bash -c '. /spack/share/spack/\
  setup-env.sh && spack location -i lammps')/bin/lmp

srun --mpi=pmi2 apptainer exec $E4S_IMAGE \
     $LMP -in in.lj -var x 2 -var y 2 -var z 2
```

Don't add --cleanenv here: the ranks find each other through PMI_* variables set by Slurm.

Hands-on 2: pull, inspect, run LAMMPS

```
$ cd $E4S_SCRATCH/e4s-labs/part2
$ apptainer pull e4s-spack-cpu_26.06.sif \
      docker://ecpe4s/e4s-spack-cpu:26.06
$ ./inspect.sh                # or explore with shell
$ sbatch lammps.sbatch
$ cat lammps-<jobid>.out
Loop time of 1.86 on 8 procs for 100 steps
Performance: ... 13.8 Matom-step/s
```

- Is HDF5 in the image? Is GROMACS built with MPI?
- Change --ntasks or the box size; compare Performance
- **Challenge:** petsc/ex50.sbatch compiles your own code against the image's PETSc

HANDS-ON **LAB_GUIDE.md · Part 2 · 25 minutes**

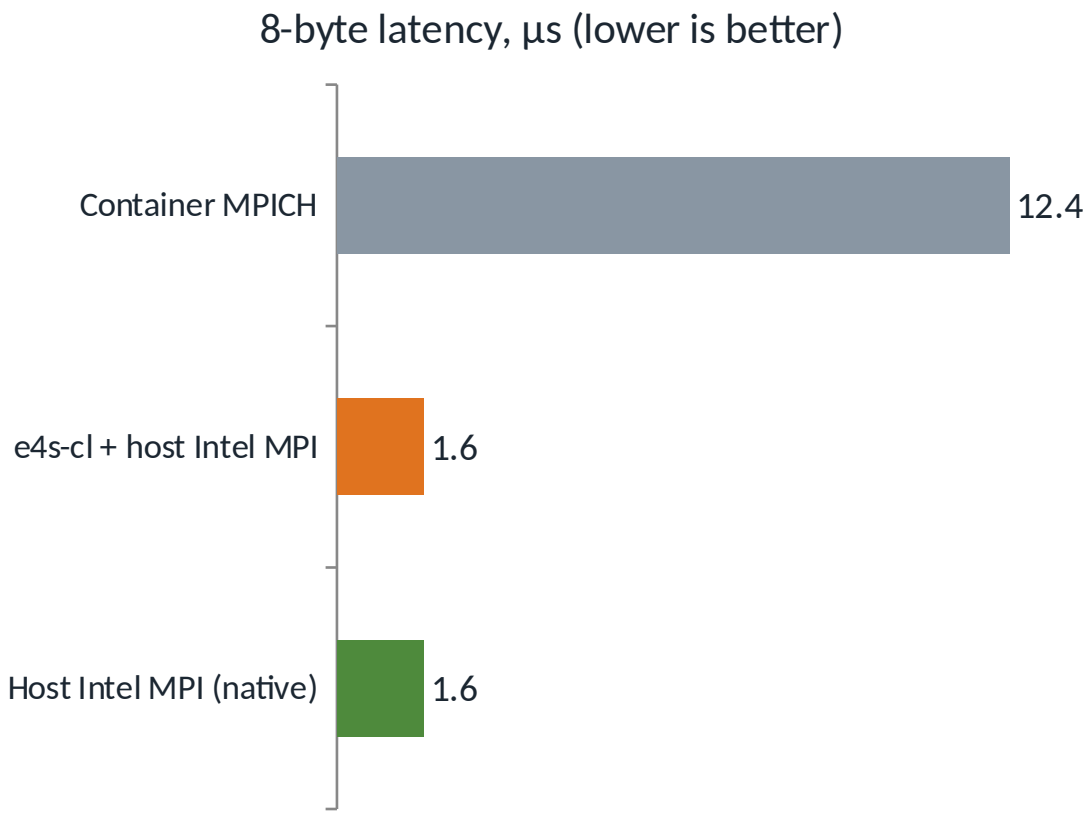
PART 3 · 45 MIN

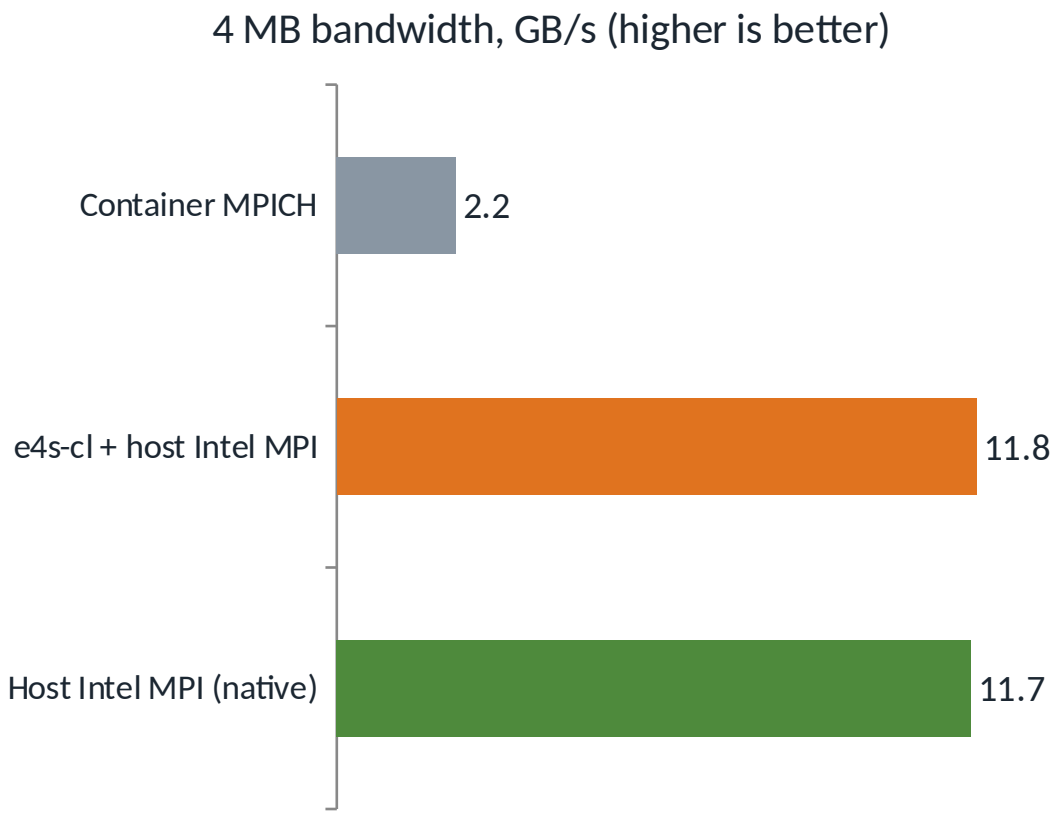
Two E4S tools: swap MPI at launch, add software to an image



e4s-cl runs a container-built MPI program on the host's MPI. e4s-alc turns a short YAML recipe into a new image.

The container's MPI does not know Midway3's network





Same container-built binary in all three bars. Ping-pong between two caslake nodes, measured Oct 24, 2026.

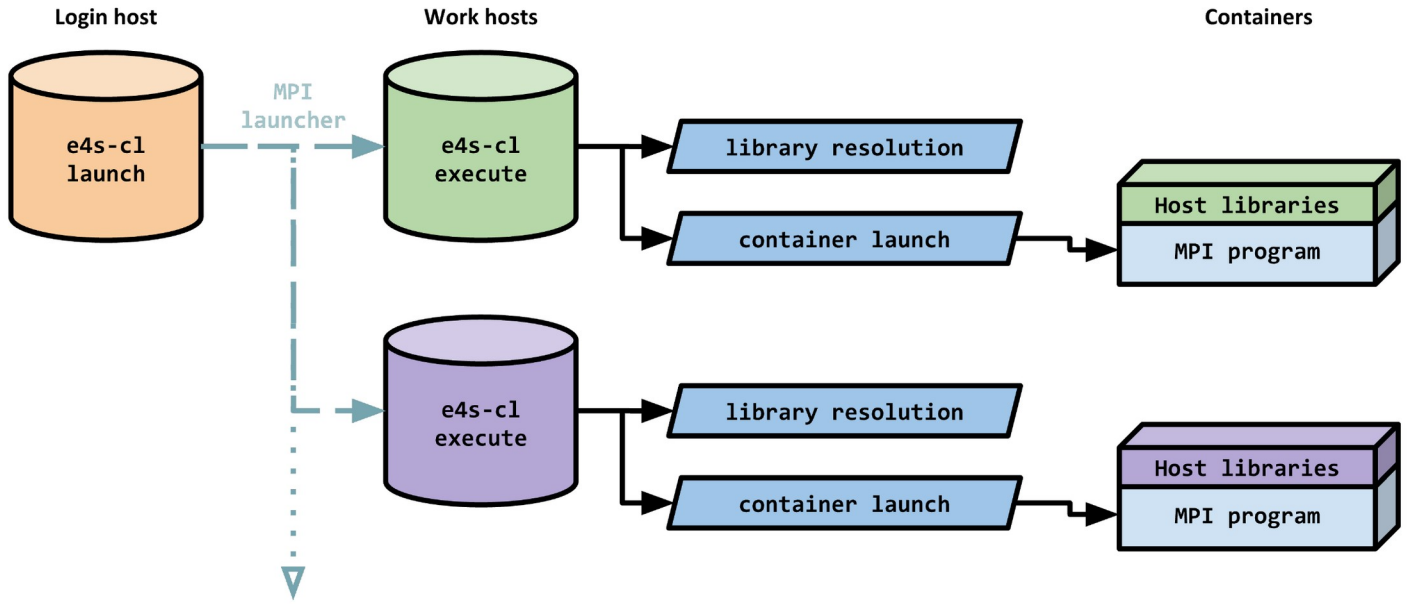
Why swapping works: the MPICH ABI

MPI implementation	Library	Swap into an MPICH-built container
MPICH (in the E4S images)	`libmpi.so.12`	**directly**
Intel MPI (Midway3: intelmpi/2021.5)	`libmpi.so.12`	**directly**
MVAPICH, Cray MPICH	`libmpi.so.12`	**directly**
Open MPI	`libmpi.so.40`	**via Wi4MPI translation (e4s-cl --from mpich)**

- **Applications and libraries stay in the container;** only the MPI library and its network stack come from the host
- **Goal:** build an MPI binary once, run it unmodified on any x86_64 cluster at native speed

After S. Shende, "Running MPI applications on other systems", E4S at NERSC, 2022; MPICH ABI initiative, mpich.org/abi.

How e4s-cl works




```
$ e4s-cl launch mpirun -np 8 -ppn 4 ./hypre_test
```

1. **Profile:** which host MPI libraries and files to inject
2. **Launch:** e4s-cl launch wraps your launcher (mpirun / srun)
3. **Bind:** on each node, host libraries are mounted at /.e4s-cl/hostlibs
4. **Preload:** LD_PRELOAD/LD_LIBRARY_PATH point libmpi.so.12 at the host's

Diagram: S. Shende, "e4s-cl Container Launcher", E4S at NERSC training, Aug 2022.

e4s-cl on Midway3

```
# usual way: trace a test program (uses ptrace)
$ module load intelmpi/2021.5
$ e4s-cl init --backend apptainer --image $E4S_IMAGE

# Midway3 disables ptrace -> build the profile by hand
$ e4s-cl profile create midway3-impi \
      --backend apptainer --image $E4S_IMAGE
$ e4s-cl profile edit midway3-impi --add-libraries \
      $I_MPI_ROOT/lib/release/libmpi.so.12 ...
$ e4s-cl profile edit midway3-impi --add-files ...
$ e4s-cl profile select midway3-impi

$ e4s-cl launch mpirun -np 2 -ppn 1 ./pingpong
```

- **make_e4scl_profile.sh** does the hand-built profile for you
- **Library list** = what Intel MPI loads on Midway3 (captured with LD_DEBUG): libmpi, libfabric + providers, verbs, mlx5, UCX, Slurm PMI
- **Launch with mpirun** (Intel MPI's hydra). e4s-cl + srun --mpi=pmi2 hung at startup in our tests

Hands-on 3a: same binary, two MPIs

```
$ cd $E4S_SCRATCH/e4s-labs/part3-mpi
$ ./compile.sh                  # inside the container
$ sbatch 01_container_mpi.sbatch
$ ./make_e4scl_profile.sh
$ sbatch 02_e4s-cl.sbatch
$ grep -A7 'MPI library' mpi-*.out
```

```
MPI library : MPICH Version: 4.3.2
           8     12.39 us       0.6 MB/s
     4194304   1883.41 us    2227.0 MB/s
MPI library : Intel(R) MPI Library 2021.5
           8      1.61 us       5.0 MB/s
     4194304    356.68 us   11759.4 MB/s
```

- The binary never changed — the reported MPI library did
- ~8× lower latency, ~5× higher bandwidth
- 02 also runs a hypre AMG solve on 8 ranks across 2 nodes

HANDS-ON **LAB_GUIDE.md · Part 3a · 15 minutes**

Adding software to an existing image: three ways



No rebuild

e4s-chain-spack.sh

Chain a Spack in your home directory to the image's Spack; spack install adds packages on top, reusing everything inside.

```
/etc/e4s/e4s-chain-spack.sh
~/spack
```



New image

e4s-alc

A YAML recipe → Apptainer definition file → apptainer build. Reproducible; share or archive the SIF.

```
e4s-alc create -f recipe.yaml
```



New image, big

Your app in the full image

Same e4s-alc recipe with the 25 GB E4S image as the base: your code compiled against the whole stack.

```
extras/e4s-plus-app.yaml
```

e4s-alc and e4s-chain-spack.sh: S. Shende, "E4S Tools", E4S 25.11 release slides.

e4s-alc: from YAML to a new image

```
# hypre.yaml
backend: singularity
bootstrap: localimage
image: .../ecpe4s-ubuntu24.04.sif
os-packages:
  - nano
post-system-stage-commands:
  - . /spack/share/spack/setup-env.sh
  - spack mirror add e4s-alc-example \
      https://cache.e4s.io/e4s-alc-example-1
  - spack buildcache keys --install --trust
  - spack install hypre
spack: False
```

e4s-alc create -f hypre.yaml
writes hypre.def

↓

apptainer build --fakeroot
hypre.sif, 1.2 GB

↓

≈ 2 minutes on 8 cores
binaries from the E4S cache, nothing compiled

Recipe adapted from the E4S/ParaTools tutorial (github.com/ParaToolsInc/e4s-cloud-examples).

Hands-on 3b: build an image, then run it with e4s-cl

```
$ cd $E4S_SCRATCH/e4s-labs/part3-alc
$ e4s-alc create -f hypre.yaml
$ less hypre.def
$ sbatch build.sbatch            # ~2 min
$ sbatch 03_run_custom.sbatch    # 2 nodes, e4s-cl
$ tail -3 alc-run-<jobid>.out
Iterations = 30
Final Relative Residual Norm = 9.525616e-09
```

- Read the generated hypre.def: which parts came from your YAML?
- **Try:** add htop to os-packages, or spack install sundials; rebuild
- **Note:** e4s-cl launch --image hypre.sif reuses the same host-MPI profile with a different image

HANDS-ON **LAB_GUIDE.md · Part 3b · 15 minutes**

Customizing the full 25 GB E4S image (demo)

```
# extras/e4s-plus-app.yaml (excerpt)
bootstrap: localimage
image: .../e4s-cpu-x86_64-26.06.sif
add-files:
  - hypre_test.c /opt/app/
os-packages:
  - htop
post-system-stage-commands:
  - spack load mpich hypre
  - cd /opt/app && mpicc ... -lHYPRE
spack: False   # keep the image's Spack
```

- **Same tool, same recipe format:** your application baked into the full stack
- **Cost:** measured ~15 min on 32 cores (2 min unpack, 12 min build) and ~160 GB of node-local disk; result 25.6 GB
- **Midway3 detail:** no /etc/subuid, so --fakeroot can't unpack files owned by other users. Unpack to a sandbox first, then build from the sandbox
- **For one-off additions,** e4s-chain-spack.sh avoids the rebuild entirely

What's next: layered E4S

OPTIONAL

AMReX sm_80 | AMReX sm_90 | heFFTe

Specialized layers: only the GPU-compiled packages, one per target

Shared provider
architecture-generic dependencies, built once

Base image
OS, compiler, MPI, CUDA

Layers compose at run time through absolute RPATHs under fixed paths, with no search-path setup. e4s-alc repacks a whole image; layering rebuilds only the layer that changed.

Midway3 and NSF ACCESS Anvil

1 of 20 buildable packages compiled per specialized layer

The rest are reused from the shared provider; the same trees run under Apptainer and Charliecloud

Each extra GPU target (Anvil)

1,412 files added, vs. 10,772 for a monolithic build

Rebuilding a layer is 1.8–2.8× faster; application performance unchanged on A100 and H100

P. Sinha, "Union of Containers: Layered Software Stacks with Cross-Container Dependency Reuse for HPC," SC26 research poster, Tue Nov 17, 2026. Manuscript in preparation with S. Shende. Not part of the hands-on.

Takeaways



Apptainer runs containers as you

No daemon, no root, one SIF file, works with Slurm.



E4S images replace installs

600+ co-tested packages; inspect variants before relying on them.



e4s-cl gives container MPI native speed

Same binary, host MPI: 1.6 µs and 11.8 GB/s across nodes.



e4s-alc customizes images

YAML recipe, binaries from the E4S cache, a new image in minutes.

Keep going

E4S	e4s.io · e4s.io/container-download · e4s.io/e4s-buildcache
E4S tools	e4s-cl.readthedocs.io · e4s-alc.readthedocs.io · github.com/E4S-Project
Examples	github.com/ParaToolsInc/e4s-cloud-examples (LAMMPS, PETSc, OSU, hypre, …)
Apptainer	apptainer.org/docs/user/latest
Today's labs	/project/rcc/workshops/e4s/labs · LAB_GUIDE.md
RCC help	help@rcc.uchicago.edu · RCC container workshops for building images



Continue at SC26

Mon Nov 16, 9:00 AM, W175a: S. Shende, “E4S: An HPC-AI Software Ecosystem for Science”

Tue Nov 17, poster session: P. Sinha, “Union of Containers” (the layered E4S work on slide 29)

Thank you

Questions?

Portions of this workshop are adapted from E4S tutorials by Sameer Shende (University of Oregon / ParaTools, Inc.) and the ParaTools e4s-cloud-examples, used with permission.

E4S is supported by DOE ASCR and the High Performance Software Foundation. Measurements: RCC Midway3, October 2026.